תרגיל 3: מילונים ועיבוד מידע

תאריך פרסום: 17.1.2024

תאריך הגשה: 23:59 2.2.2024

מתרגלת אחראית:

משקל תרגיל: 3 נקודות

מטרות העבודה: תרגול עבודה עם מחרוזות (str) ומילונים לשם עיבוד מידע.

הנחיות לעבודה 3:

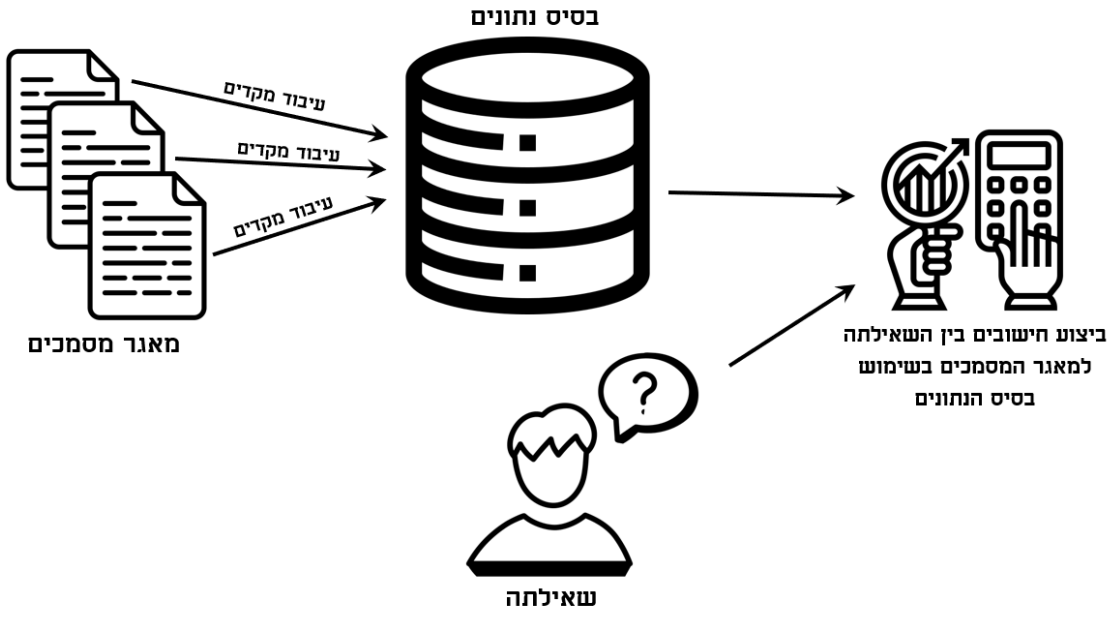
- קראו בקפידה את הנחיות ההגשה. אל תגישו קבצים או קוד מיותר. וודאו כי אתם עומדים בכל ההנחיות לפני ההגשה!
- מומלץ לקרוא את כל העבודה בשלמותה לפני תחילת כתיבת קוד הפתרון.
- אין להשתמש בספריות חיצוניות (ניתן להשתמש בספריה math).
- ניתן להוסיף פונקציות עזר. האחריות על נכונות הקוד בפונקציות אלה היא עליכם.
- חובה לכתוב docstring (כפי שנלמד בכיתה) לכל פונקציה.

- **יש להגיש קובץ אחד בשם HW3.py (שלד הקובץ נתון לכם ורצוי למלא את הפתרון במקומות המתאימים).**

רקע ומטרת העבודה:

בתרגיל זה תתנסו בעבודה עם מחרוזות ומילונים. בעזרתם, תממשו תוכנית המבצעת עיבוד מקדים של מאגר מסמכים לבסיס נתונים, ומנוע חיפוש בו ניתן לחפש מחרוזת בבסיס הנתונים באמצעות שאילתות. בחלק הראשון של התרגיל, תממשו עיבוד מקדים שנועד להסיר מילים ותווים לא הכרחיים, לשנות את המילים במסמך לצורה יותר אחידה ולבצע הפרדה של המילים. בחלק השני, תאגדו את המידע מכל המסמכים במאגר לבסיס נתונים שיאפשר חיפוש מהיר. בחלק השלישי, תשתמשו בבסיס הנתונים שיצרתם על מנת לבצע חישובים שונים המערבים שאילתה (מחרוזת) ואת מאגר המסמכים. בחלק הרביעי והאחרון, תאחדו את כל השלבים לתפריט אחד בו המשתמש יוכל לבצע פעולות על מאגר מסמכים.

להלן תרשים:

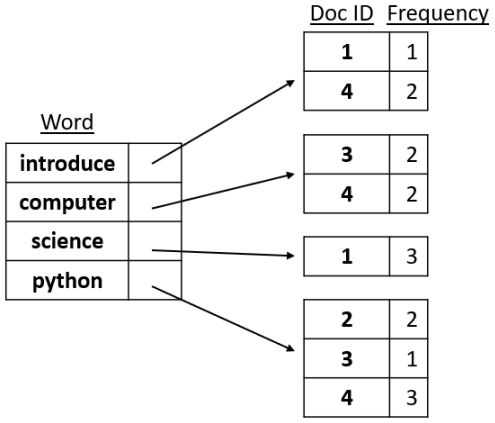


<u>כמה הגדרות חשובות לפני שמתחילים:</u>

- **Stop words** – רשימת מילים המושמטות מהמסמכים מכיוון שהן לא מוסיפות מידע למשפט. בדרך כלל נכללות ברשימה זו מילים שהן מילות קישור, מילות יחס, כינויי גוף וכדומה. המשתנה הגלובלי שסופק לכם "stop_words" מכיל רשימה של כל המילים הנחשבות כ-stop word בעבודה זו ואין לשנות אותה.
- **Punctuations** – סימני פיסוק. המשתנה הגלובלי "punctuations" הינו רשימה המכילה את כל סימני הפיסוק היכולים להופיע במאגר המסמכים בעבודה ואין לשנות רשימה זו.
- **Corpus** – אוסף מסמכים, קורפוס. מילון כאשר המפתחות הינם המספר המזהה של המסמך והערכים הינם המילים מהמסמך כמחרוזת. המשתנה corpus <u>אינו</u> משתנה גלובלי ויש להעביר אותו כארגומנט

בכל קריאה לפונקציה המשתמשת בו. שימו לב שבבדיקת העבודה נשתמש בקורפוס אחר מזה שנתון לכם בקובץ העבודה.

- **Stemming** – העברת מילים לצורת הבסיס שלהן. הפעולה מתבצעת על ידי הסרה של סיומות מהמילה. לדוגמא לאחר stemming המילה chicken והמילה chickens יחשבו כאותה המילה (chicken).
- **Inverted index** – מבנה נתונים בו עבור כל מילה המופיעה במאגר המסמכים נשמרים כל המסמכים בהם המילה הופיעה. המימוש הוא באמצעות מילון כאשר המפתחות הינם מילים המופיעות במאגר המסמכים (מטיפוס מחרוזת). כל מפתח ממופה למילון כאשר המפתחות שלו הם המספרים המזהים של מסמכים המכילים את המילה, והערכים הינם מספר המופעים של מילת הקלט במסמך.



על פי התרשים, המילה introduce מופיעה במסמך #1, פעם אחת, ובמסמך #4, פעמיים.

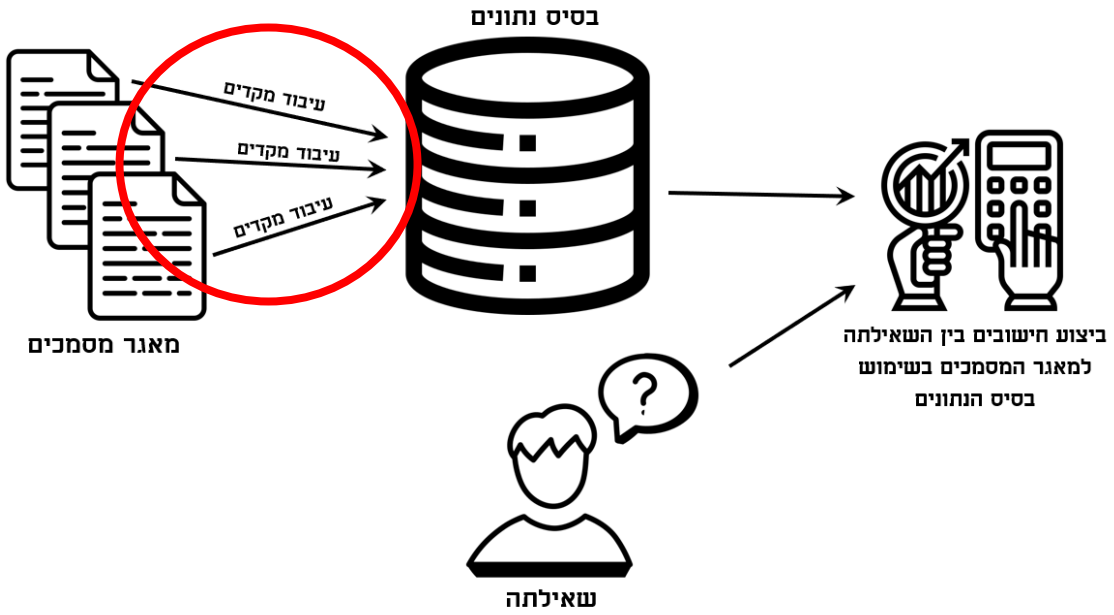
- **TF-IDF**–קיצור של term frequency–inverse document frequency, בעברית: תדירות מונח (tf) - תדירות מסמכים הופכית (idf) שמשמעותו ציון מספרי שנועד לשקף את החשיבות של מילה מסוימת למסמך מסוים.

תדירות מונח (tf) – באיזה תדירות מופיעה המילה במסמך.

תדירות מסמכים הופעכית (idf) – חשיבות המילה בכל מאגר המסמכים.

חלק 1 – עיבוד מקדים

בחלק זה תממשו חמש פונקציות עיבוד מקדים, כלומר הכנה של מסמך להמשך עיבוד, ופונקציה נוספת אשר מאחדת את כל חמשת השלבים.



def remove_punctuations(text)

הפונקציה מקבלת כקלט מחרוזת (str) ומחזירה מחרוזת חדשה (str) התואמת לקלט, לאחר הסרת סימני פיסוק. כל סימן פיסוק בקלט יוחלף ברווח בפלט. סימני הפיסוק מוגדרים ברשימה punctuations.

דוגמת הרצה:

```
>>> remove_punctuations("Hi,I'm here!")
'Hi I m here '
```

def remove_digits(text)

הפונקציה מקבלת כקלט מסמך (str) ומחזירה מסמך חדש (str) התואם לקלט, ללא מספרים. כל ספרה בקלט תוחלף בתו ריק בפלט.

דוגמת הרצה:

```
>>> remove_digits("Hello 2024, goodbye 2023")
'Hello , goodbye '
```

def remove_spaces(text)

הפונקציה מקבלת בקלט מסמך (str) ומחזירה מסמך חדש (str) התואם לקלט, כאשר כל רצף של רווחים יוחלף ברווח בודד.

דוגמת הרצה:

```
>>> remove_spaces("hi,      i want     to join")
'hi, i want to join'
```

def remove_stopwords(words_list)

הפונקציה מקבלת בקלט רשימה של מילים ומחזירה רשימה חדשה התואמת לקלט, אך ללא מילות stop words, שמוגדרות ברשימה stop_words.

דוגמת הרצה:

```
>>> remove_stopwords(['I', 'can', 'love', 'me', 'better', 'than', 'you', 'can'])
['can', 'love', 'me', 'better', 'than', 'can']
```

המילים שהוסרו הינן: 'I', 'you' מכיוון שהן נמצאות ברשימה stop_words.

def stemming(words_list)

הפונקציה מקבלת בקלט רשימה של מילים ומחזירה רשימה חדשה התואמת לקלט, כאשר כל מילה מומרת לצורת הבסיס שלה. על מנת להמיר מילה לצורת הבסיס יש לשנות את סוף המילה לפי הכללים הבאים:

Rule	Example
ies -> y	ponies -> pony
sses -> ss	accesses -> access
s ->	cats -> cat
ed ->	talked -> talk
ing ->	working -> work

שימו לב:

- סדר הפעלת הכללים נקבע לפי הסדר בטבלה.
- הניחו כי יש להמיר כל מילה לכל היותר פעם אחת.
- תהליך ה-stemming לא בהכרח יוצר מילים אמיתיות, זה בסדר.

דוגמת הרצה:

```
>>>  stemming(['hoodies', 'glasses', 'kids', 'knocked', 'reading'])
['hoody', 'glass', 'kid', 'knock', 'read']
```

def preprocessing(text)

פונקציה המאחדת את תהליך העיבוד המקדים של מסמך. הפונקציה מקבלת כקלט מסמך (str) ומחזירה רשימת מילים לאחר עיבוד מקדים.

הפונקציה מבצעת את השלבים הבאים (בסדר זה):

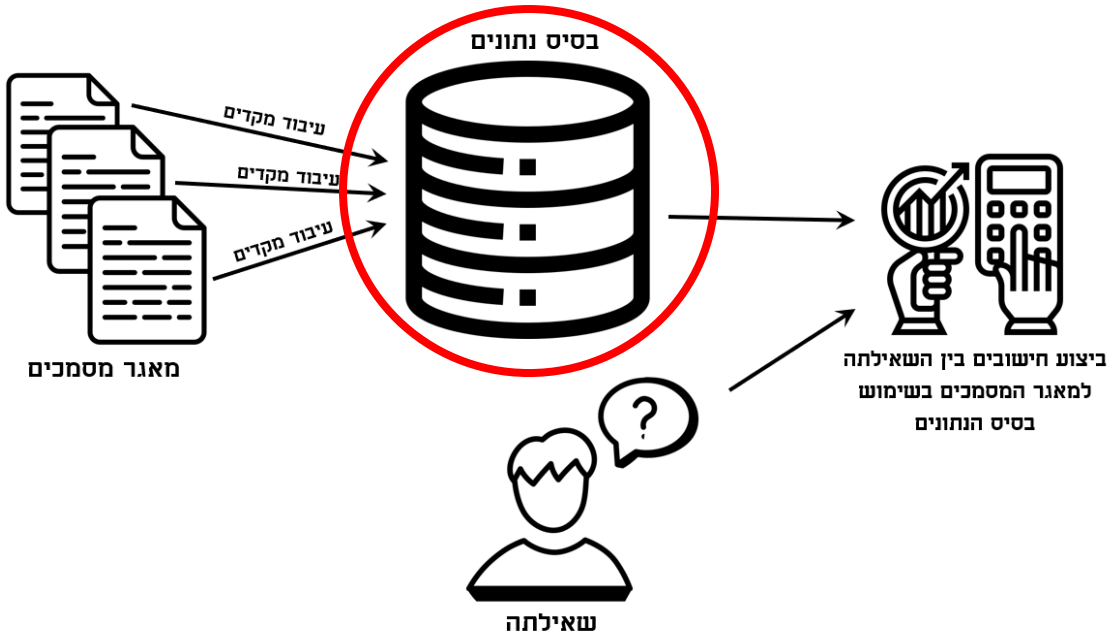
- הסרת סימני פיסוק.
- הסרת ספרות.
- הסרת רווחים מיותרים.
- המרת אותיות "גדולות" (capital) לקטנות (lower case)
- פיצול המסמך לרשימת מילים, ההפרדה בין מילים נקבעת לפי קיום של רווח.
- הסרה של stop words.
- ביצוע stemming.
- הסרה של מחרוזות ריקות (במידה ויש '' ברשימה).

דוגמת הרצה:

```
>>>preprocessing("Yesterday,    I received 3 emails from    my colleagues!")

['yesterday', 'receiv', 'email', 'colleague']
```

חלק 2 – יצירת בסיס נתונים למאגר מסמכים



בחלק זה תממשו פונקציות היוצרות ממאגר מסמכים בסיס הנתונים ומתחזקות אותו. בסיס הנתונים מאפשר ייצוג דחוס של מאגר המסמכים (על מנת לחסוך בזיכרון) וחיפוש מהיר (ביחס לחיפוש במאגר המסמכים המקורי).

בסיס הנתונים מורכב משני מילונים:

- documents data – מילון השומר עבור כל מסמך את אורכו (מספר המילים במסמך).
- inverted index – מילון השומר עבור כל מילה את מספר המופעים שלה בכל מסמך (ראו איור בהגדרות בעמוד 3).

דוגמאות הריצה בחלק זה ובחלק הבא של התרגיל יסתמכו על מאגר המסמכים הבא:

1: The cat played the piano

2: 5 cats are playing ball in the backyard!

3: The grand piano is in the house

def get_documents_data(corpus)

פונקציה היוצרת documents data עבור מאגר מסמכים. הפונקציה מקבלת כקלט מילון המייצג מאגר מסמכים, כאשר המפתחות הינם המספרים המזהים של המסמך (int) והערכים הינם המחרוזות המייצגות את המסמכים התואמים (str). הפונקציה מחזירה documents_data - מילון אשר המפתחות בו הינם המספר המזהה של כל מסמך (int) והערכים הינם מספר המילים במסמך <u>לאחר</u> תהליך העיבוד המקדים מחלק 1 (int).

דוגמת הרצה:

```
>>> get_documents_data(corpus_example)

{1: 3, 2: 4, 3: 3}
```

הסבר למסמך מספר 1: המילה "the" מופיעה ברשימת המילים stop words ולכן לאחר העיבוד המקדים מסמך זה מכיל 3 מילים.

def create_inverted_index(corpus)

פונקציה היוצרת inverted index עבור מאגר מסמכים. הפונקציה מקבלת כקלט מילון המייצג מאגר מסמכים, כאשר המפתחות הינם המספר המזהה של המסמך (int) והערכים הינם המחרוזות המייצגות את המסמך (str). הפונקציה מחזירה inverted index של מאגר המסמכים לאחר עיבוד מקדים. inverted index הינו מילון, כאשר המפתחות בו הם המילים המרכיבות את מאגר המסמכים לאחר עיבוד מקדים. הערך עבור כל מילה הינו מילון אשר המפתחות בו הינם המסמכים אשר מכילים את המילה והערכים הינם מספר המופעים של המילה בכל מסמך בהתאמה (ראו איור בעמוד 3).

דוגמת הרצה:

```
create_inverted_index(corpus_example)<<<

{'cat': {1: 1, 2: 1}, 'play': {1: 1, 2: 1}, 'piano': {1: 1, 3: 1}, 'ball': {2: 1}, 'backyard': {2: 1}, 'grand': {3:
1}, 'house': {3: 1}}
```

הסבר על המילה 'play' – המילה 'play' מופיעה במסמך אחד בצורה 'played' ובמסמך 2 בצורה ' playing'. לאחר עיבוד מקדים שתי המילים יופיעו כ-'play', פעם אחת במסמך 1 ופעם אחת במסמך 2. המילה 'play' אינה מופיעה במסמך 3 ולכן הוא אינו מופיע במילון המתאים למילה 'play'.

def add_to_data(inverted_index, documents_data, doc_id, text)

פונקציה המוסיפה מסמך חדש לבסיס הנתונים. הפונקציה מקבלת כקלט *inverted_index*, *documents_data*, מספר מזהה של המסמך החדש (int) ומחרוזת של המסמך החדש (str) ומחזירה *inverted_index* ו- *documents_data* (בסדר הזה) מעודכנים לפי המסמך החדש.

שימו לב:

- הניחו כי המספר המזהה של המסמך החדש אינו מופיע כבר בבסיס הנתונים.
- הפונקציה אינה משנה את מאגר המסמכים המקורי אלא רק את המילונים *inverted_index* ו- *documents_data*.

דוגמת הרצה:

```
new_inverted_index, new_documents_data = add_to_data(inverted_index, documents_data, 4,
                                                     'The dog is sleeping in the backyard')
print(f'inverted_index: {new_inverted_index}')
print(f'documents_data: {new_documents_data}')
```

inverted_index: {'cat': {1: 1, 2: 1}, 'play': {1: 1, 2: 1}, 'piano': {1: 1, 3: 1}, 'ball': {2: 1}, 'backyard': {2: 1, 4: 1}, 'grand': {3: 1}, 'house': {3: 1}, 'dog': {4: 1}, 'sleep': {4: 1}}

documents_data: {1: 3, 2: 4, 3: 3, 4: 3}

def remove_from_data(inverted_index, documents_data, doc_id)

פונקציה המוחקת מסמך מבסיס הנתונים. הפונקציה מקבלת בקלט *inverted_index*, *documents_data* ומספר מזהה של המסמך אותו רוצים למחוק (int) ומחזירה *inverted_index* ו- *documents_data* (בסדר הזה) מעודכנים ללא המסמך שנבחר למחיקה.

שימו לב:

- הניחו כי המספר המזהה של המסמך קיים במאגר הנתונים.
- הפונקציה אינה משנה את מאגר הנתונים המקורי אלא רק את המילונים *inverted_index*ו- *documents_data*.
- אם לאחר מחיקת המסמך במילון *inverted_index* יש מילים שהמילון בערך שלהן נותר ריק, יש למחוק מילים אלו מהמילון *inverted_index*.

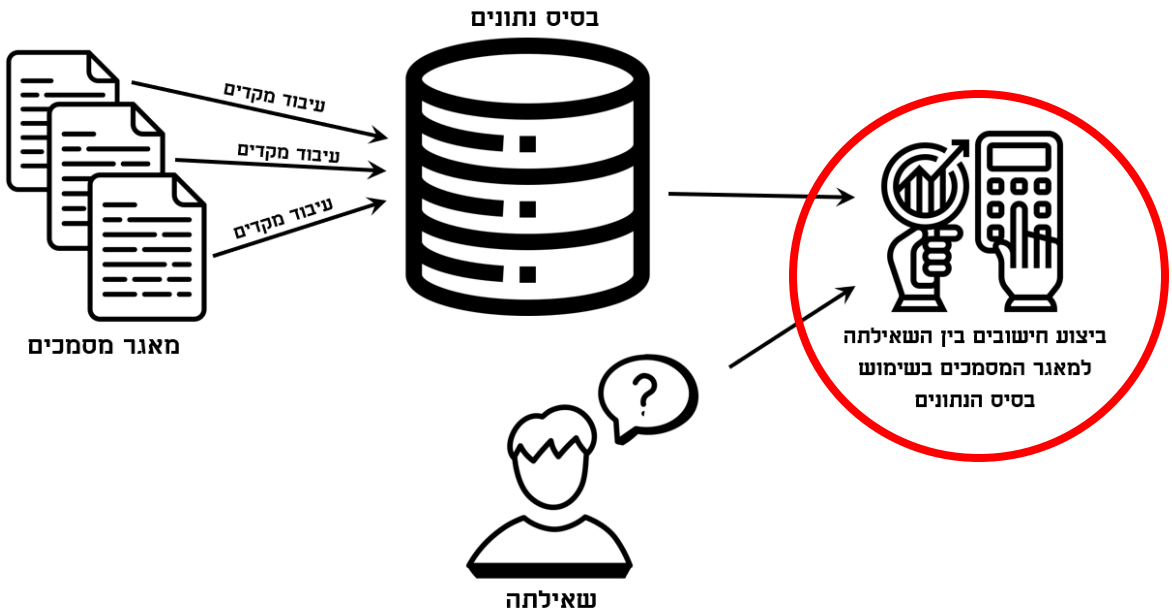
דוגמת הרצה (על המאגר המקורי):

```
new_inverted_index, new_documents_data = remove_from_data(inverted_index, documents_data, 2)
print(f'inverted_index: {new_inverted_index}')
print(f'documents_data: {new_documents_data}')
```

inverted_index: {'cat': {1: 1}, 'play': {1: 1}, 'piano': {1: 1, 3: 1}, 'grand': {3: 1}, 'house': {3: 1}}

documents_data: {1: 3, 3: 3}

חלק 3 – חישוב TF-IDF



בחלק זה תממשו שתי פונקציות לחישוב TF-IDF בין שאילתה למסמכים בבסיס הנתונים. שאילתה הינה מחרוזת אותה מזין המשתמש על מנת לחפש במאגר מסמכים. ערך ה- TF-IDF של מסמך בהקשר של שאילתה יקבע עד כמה המסמך רלוונטי לבקשת המשתמש.

def calculate_tf_idf(word, doc, inverted_index, documents_data):

פונקציה המחשבת את ערך ה-TF-IDF של מסמך מבסיס הנתונים ביחס לשאילתה. הפונקציה מקבלת כקלט מילה 'word' (str), מספר מזהה של מסמך 'doc' (int) , inverted_index ו-documents_data ומחזירה את ערך ה-TF-IDF של המילה 'word' בהקשר של המסמך 'doc'.

ערך ה-TF-IDF מחושב באופן הבא:

$$IDF = \log_2\left(\frac{Total\ number\ of\ documents\ in\ the\ corpus}{Number\ of\ documents\ containing\ the\ term\ word}\right)$$

$$TF = \frac{Number\ of\ times\ term\ word\ appears\ in\ document\ doc}{Total\ number\ of\ terms\ in\ document\ doc}$$

$$TF - IDF = TF \cdot IDF$$

שימו לב:

- הערך המוחזר צריך להיות מעוגל ל-3 ספרות אחרי הנקודה.
- הניחו כי המספר המזהה של המסמך קיים במאגר הנתונים.
- הניחו כי המילה קיימת במילון inverted index.

דוגמת הרצה:

```
documents_data = get_documents_data(corpus_example)
inverted_index = create_inverted_index(corpus_example)
tf_idf_cat = calculate_tf_idf('cat', 1, inverted_index, documents_data)
print(f'TF-IDF of doc 1 and the word cat is: {tf_idf_cat}')
tf_idf_ball = calculate_tf_idf('ball', 1, inverted_index, documents_data)
print(f'TF-IDF of doc 1 and the word ball is: {tf_idf_ball}')
```

TF-IDF of doc 1 and word cat is: 0.195

TF-IDF of doc 1 and word ball is: 0.0

חישוב עבור המילה cat ומסמך 1:

$$Total\ number\ of\ documents\ in\ the\ corpus = 3$$

$$Number\ of\ documents\ containing\ the\ term\ 'cat' = 2$$

$$Number\ of\ times\ term\ 'cat' appears\ in\ document\ 1 = 1$$

$$Total\ number\ of\ terms\ in\ document\ 1 = 3$$

$$TF - IDF('cat', 1) = \log_2\left(\frac{3}{2}\right) \cdot \frac{1}{3} = 0.195$$

def get_scores_of_relevance_docs(query, inverted_index, documents_data):

הפונקציה מקבלת כקלט שאילתה (query) שהינה <u>רשימת</u> מילים, inverted_index ו-documents_data ומחזירה מילון כאשר המפתחות הינם המספר המזהה של המסמכים הרלוונטים והערכים הינם ציון ה-TF-IDF בין מילות השאילתה למסמך.

הגדרות חשובות:

- מסמך רלוונטי – מסמך שלפחות אחת מהמילים המופיעות בשאילתה מופיעות בו.
- TF-IDF בין רשימת מילים למסמך הינו סכום ה-TF-IDF של המילים ברשימה למסמך.

שימו לב:

- <u>לא ניתן</u> להניח כי כל המילים המופיעות ברשימת המילים של השאילתה מופיעות בבסיס הנתונים. במידה ויש מילה שלא מופיעה בבסיס הנתונים ניתן לדלג עליה.
- ניתן להניח כי רשימת המילים בשאילתה הינה לאחר תהליך עיבוד מקדים לפי חלק 1.

דוגמת הרצה:

```
documents_data = get_documents_data(corpus_example)
inverted_index = create_inverted_index(corpus_example)
print(get_scores_of_relevance_docs(['play', 'cat', 'blue'], inverted_index, documents_data))
```

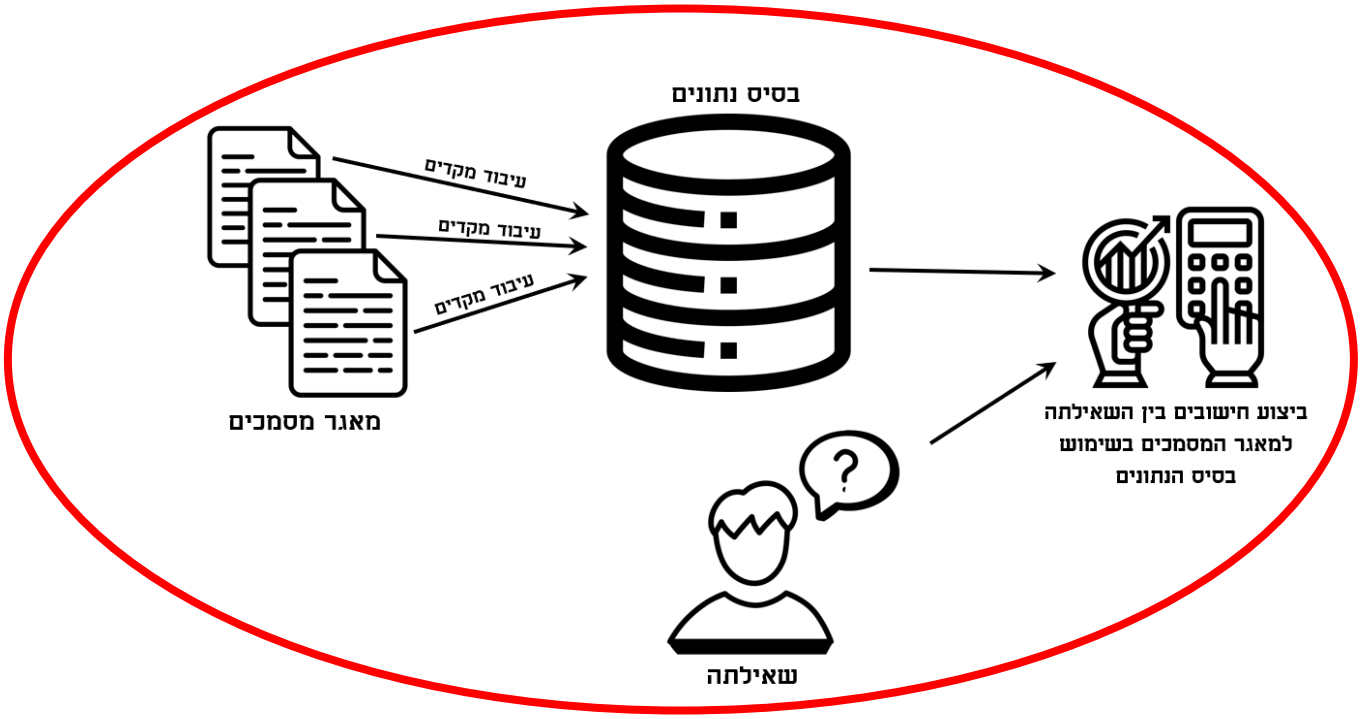
{1: 0.39, 2: 0.292}

שימו לב שמסמך מספר 3 לא מופיע בפלט מכיוון שאין לו אף מילה משותפת עם מילות השאילתה.

```
print(get_scores_of_relevance_docs(['grand', 'fish', 'piano'], inverted_index, documents_data))
```

{3: 0.723, 1: 0.195}

חלק 4 – תפריט מנוע חיפוש



בחלק זה תממשו פונקציה המאחדת את כל השלבים שביצעתם עד כה ותהווה תפריט למנוע חיפוש.

def menu(corpus)

הפונקציה מקבלת כקלט מאגר מסמכים (מילון שהמפתחות בו הם המספרים המזהים של המסמכים והערכים הינם המחרוזות של כל מסמך) ואינה מחזירה דבר.

תחילה הפונקציה יוצרת את בסיס הנתונים, כלומר יוצרת את המילונים inverted_index ו-documents_data עבור מאגר המסמכים שבקלט, corpus.

לאחר מכן יוצג למשתמש התפריט הבא:

Choose an option from the menu:

(1) Insert a query.

(2) Add document to corpus.

(3) Calculate TF-IDF Score for a word in a document.

(4) Delete a document from the corpus.

(5) Exit.

Your choice:

שימו לב שהטקסט נתון לכם בקובץ התרגיל (שימו לב כי על הטקסט המלא להופיע יחד בבקשת הקלט מהמשתמש, כפי שנתון המשתנה choice בקובץ התרגיל, ולא כהדפסה של התפריט בעזרת print ולאחר מכן בקשת הקלט).

<u>בחירה באפשרות (1):</u>

כאשר המשתמש יבחר באפשרות זו תוצג לו ההודעה הבאה:

Write your query here:

כלומר על המשתמש להכניס שאילתה. שאילתה הינה מחרוזת חופשית.

עליכם לבצע עיבוד מקדים לשאילתה אותה הכניס המשתמש ולקבל רשימת מילים (חלק 1).

לאחר מכן יוצג למשתמש ההודעה הבאה:

Choose the type of results you would like to retrieve:

(A) All relevant documents.

(B) The most relevant document.

(C) Back to the main menu.

Your choice:

שימו לב שהטקסט נתון לכם בקובץ התרגיל. (שימו לב כי על הטקסט המלא להופיע יחד בבקשת הקלט מהמשתמש, כפי שנתון במשתנה query_choice בקובץ התרגיל, ולא כהדפסה של התפריט בעזרת print ולאחר מכן בקשת הקלט).

בחירה באפשרות (A): יודפסו למשתמש כל המסמכים הרלוונטים לשאילתה שהכניס וציון ה-TF-IDF בינם לבין השאילתה. המסמכים יודפסו אחד אחרי השני (בשורה חדשה) בפורמט הבא: *{ doc_id} : {score}}* המסמכים יודפסו לפי סדר ה-ID של המסמכים מהקטן לגדול.

בחירה באפשרות (B): יודפס למשתמש מספר המסמך עם ציון ה-TF-IDF הגבוה ביותר, כלומר המסמך שהכי תואם לשאילתה. תודפס ההודעה הבאה:

The most relevant document is {highest_score_doc} with a score of {highest_score}

בחירה באפשרות (C): המשתמש יחזור לתפריט הראשי והתפריט יודפס בפניו.

בחירת אפשרות שלא מתוך האפשרויות הנתונות: תודפס למשתמש ההודעה הבאה:

Invalid choice. Please select a valid option.

לאחר מכן יודפס התפריט מחדש.

<u>בחירה באפשרות (2):</u>

הוספת מסמך לבסיס הנתונים.

תחילה נבקש מהמשתמש את מספר הזהות של המסמך החדש:

Insert the document ID:

ניתן להניח כי המשתמש הכניס מספר שלם אך אין להניח כי הוא הכניס מספר שלא קיים בבסיס הנתונים. במידה והמשתמש הכניס מספר שכבר קיים יש להציג לו את ההודעה הבאה:

The document ID {doc_id} is already in corpus.

לאחר מכן יש לחזור להודעה הקודמת על מנת לבקש מספר חדש.

כאשר המשתמש הכניס מספר שלא קיים בבסיס הנתונים יש לבקש מהמשתמש את הטקסט של המסמך באופן הבא:

Insert the text of the document:

לאחר מכן יש להוסיף את המסמך החדש למילונים inverted_index ו-documents_data ולהדפיס למשתמש הודעה שההוספה הושלמה:

Document {doc_id} was successfully added!

שימו לב שהוספת המסמך החדש לא משנה את מאגר המסמכים המקורי אלא רק את המילונים *inverted_index* ו-*documents_data*.

<u>בחירה באפשרות (3):</u>

חישוב את ערך ה-TF-IDF בין מילה למסמך.

תחילה נבקש מהמשתמש את המספר המזהה של המסמך המבוקש:

Insert the document ID:

ניתן להניח כי המשתמש הכניס מספר שלם אך אין להניח כי הוא הכניס מספר שקיים בבסיס הנתונים. במידה והמשתמש הכניס מספר שלא קיים יש להציג לו את ההודעה הבאה:

The document ID {doc_id} is not in corpus.

לאחר מכן יש לחזור להודעה הקודמת על מנת לבקש מספר חדש.

כאשר המשתמש הכניס מספר שקיים בבסיס הנתונים יש לבקש מהמשתמש את המילה שירצה לחשב את ה-TF-IDF בינה לבין המסמך:

Insert a word:

אם המילה לא נמצאת בבסיס הנתונים (לאחר עיבוד מקדים של המילה) יש להדפיס את ההודעה הבאה:

The word { word_after_preprocces } is not in corpus.

לאחר מכן יש לחזור להודעה הקודמת על מנת לבקש מילה חדשה.

כאשר המשתמש הכניס מילה שקיימת בבסיס הנתונים יש לחשב את ערך ה-TF-IDF בינה (לאחר עיבוד מקדים) לבין המסמך שנבחר ולהדפיס את ההודעה הבאה:

TF-IDF of the word {word_after_preprocces} in document {doc_id} is: {tf_idf}

הערה: עיבוד מקדים של מילה בודדת במקרה זה הינו הפיכת המילה לאותיות קטנות וביצוע stemming בלבד.

<u>בחירה באפשרות (4):</u>

מחיקה של מסמך מבסיס הנתונים.

תחילה נבקש מהמשתמש את המספר המזהה של המסמך המבוקש:

Insert the document ID:

ניתן להניח כי המשתמש הכניס מספר שלם אך <u>אין</u> להניח כי הוא הכניס מספר שקיים בבסיס הנתונים. במידה והמשתמש הכניס מספר שלא קיים יש להציג לו את ההודעה הבאה:

The document ID {doc_id} is not in corpus.

לאחר מכן יש לחזור להודעה הקודמת על מנת לבקש מספר חדש.

כאשר המשתמש הכניס מספר שקיים בבסיס הנתונים יש למחוק את מסמך זה מהמילונים inverted_index ו-documents_data ולהדפיס למשתמש הודעה שהמחיקה הושלמה:

Document {doc_id} is successfully deleted.

שימו לב שמחיקת המסמך לא משנה את מאגר המסמכים המקורי אלא רק את בסיס הנתונים, כלומר את המילונים *inverted_index* ו-*documents_data*.

<u>בחירה באפשרות (5):</u>

אפשרות זו תסיים את ריצת הפונקציה.

<u>בחירה באפשרות לא חוקית:</u>

תודפס למשתמש ההודעה הבאה:

Invalid choice. Please select a valid option.

לאחר מכן יודפס התפריט מחדש.

<u>שימו לב:</u> דוגמת הרצה מפורטת נמצאת בקובץ menu_example.txt. מומלץ להריץ את הקוד שלכם לפי השלבים בדוגמא ולהשתמש ב-text compare על מנת לוודא שההדפסות שלכם תואמות להדפסות שבדוגמא.

כל ההדפסות צריכות להיות זהות לדוגמא שלנו ברמת הרווחים, אותיות קטנות/גדולות וכדומה. אחרת הטסטים לא יעברו.

בהצלחה!